

管理服务

管理服务

本章介绍如何使用 systemd 进行系统和服务管理。

- 管理服务
 - 简介
 - 概念介绍
 - 特性说明
 - 更快的启动速度
 - 提供按需启动能力
 - 采用 cgroup 特性跟踪和管理进程的生命周期
 - 启动挂载点和自动挂载的管理
 - 实现事务性依赖关系管理
 - 与 SysV 初始化脚本兼容
 - 能够对系统进行快照和恢复
 - 管理系统服务
 - sysvinit 命令和 systemd 命令
 - 显示所有当前服务
 - 显示服务状态
 - 运行服务
 - 关闭服务
 - 重启服务
 - 启用服务
 - 禁用服务
 - 改变运行级别
 - Target 和运行级别
 - 查看系统默认启动目标
 - 查看当前系统所有的启动目标
 - 改变默认目标
 - 改变当前目标

- 切换到救援模式
- 切换到紧急模式
- 关闭、暂停和休眠系统
 - systemctl 命令
 - 关闭系统
 - 重启系统
 - 使系统待机
 - 使系统休眠

简介

systemd 是在 Linux 下，与 SysV 和 LSB 初始化脚本兼容的系统和服務管理器。systemd 使用 socket 和 D-Bus 来开启服务，提供基于守护进程的按需启动策略，支持快照和系统状态恢复，维护挂载和自挂载点，实现了各服务间基于从属关系的一个更为精细的逻辑控制，拥有更高的并行性能。

概念介绍

systemd 开启和监督整个系统是基于 unit 的概念。unit 是由一个与配置文件对应的名字和类型组成的（例如：avahi.service unit 有一个具有相同名字的配置文件，是守护进程 Avahi 的一个封装单元）。unit 有多重类型，如表 1 所示。

表 1 unit 说明

unit 名称

后缀名

描述

Service unit

.service

系统服务。

Target unit

.target

一组 systemd units。

Automount unit

.automount

文件系统挂载点。

Device unit

.device

内核识别的设备文件。

Mount unit

.mount

文件系统挂载点。

Path unit

.path

在一个文件系统中的文件或目录。

Scope unit

.scope

外部创建的进程。

Slice unit

.slice

一组用于管理系统进程分层组织的 units。

Socket unit

.socket

一个进程间通信的 Socket。

Swap unit

.swap

swap 设备或者 swap 文件。

Timer unit

.timer

systemd 计时器。

所有的可用 systemd unit 类型，可在如表 2 所示的路径下查看。

表 2 可用 systemd unit 类型

路径

描述

/usr/lib/systemd/system/

随安装的 RPM 产生的 systemd units。

/run/systemd/system/

在运行时创建 systemd units。

/etc/systemd/system/

由系统管理员创建和管理的 systemd units。

特性说明

更快的启动速度

systemd 提供了比 UpStart 更激进的并行启动能力，采用了 socket/D-Bus activation 等技术启动服务，带来了更快的启动速度。

为了减少系统启动时间，systemd 的目标是：

- 尽可能启动更少的进程。
- 尽可能将更多进程并行启动。

提供按需启动能力

当 sysvinit 系统初始化的时候，它会将所有可能用到的后台服务进程全部启动运行。并且系统必须等待所有的服务都启动就绪之后，才允许用户登录。这种做法有两个缺点：首先是启动时间过长；其次是系统资源浪费。

某些服务很可能在很长一段时间内，甚至整个服务器运行期间都没有被使用过。比如 CUPS，打印服务在多数服务器上很少被真正使用到。您可能没有想到，在很多服务器上

SSHD 也是很少被真正访问到的。花费在启动这些服务上的时间是不必要的；同样，花费在这些服务上的系统资源也是一种浪费。

systemd 可以提供按需启动的能力，只有在某个服务被真正请求的时候才启动它。当该服务结束，systemd 可以关闭它，等待下次需要时再次启动它。

采用 cgroup 特性跟踪和管理进程的生命周期

init 系统的一个重要职责就是负责跟踪和管理服务进程的生命周期。它不仅可以启动一个服务，也能够停止服务。这看上去没有什么特别的，然而在真正用代码实现的时候，您或许会发现停止服务比一开始想的要困难。

服务进程一般都会作为守护进程（daemon）在后台运行，为此服务程序有时候会派生（fork）两次。在 UpStart 中，需要在配置文件中正确地配置 expect 小节。这样 UpStart 通过对 fork 系统调用进行计数，从而获知真正的精灵进程的 PID 号。

cgroup 已经出现了很久，它主要用来实现系统资源配额管理。cgroup 提供了类似文件系统的接口，使用方便。当进程创建子进程时，子进程会继承父进程的 cgroup。因此无论服务如何启动新的子进程，所有的这些相关进程都会属于同一个 cgroup，systemd 只需要简单地遍历指定的 cgroup 即可正确地找到所有的相关进程，将它们逐一停止即可。

启动挂载点和自动挂载的管理

传统的 Linux 系统中，用户可以用/etc/fstab 文件来维护固定的文件系统挂载点。这些挂载点在系统启动过程中被自动挂载，一旦启动过程结束，这些挂载点就会确保存在。这些挂载点都是对系统运行至关重要的文件系统，比如 HOME 目录。和 sysvinit 一样，systemd 管理这些挂载点，以便能够在系统启动时自动挂载它们。systemd 还兼容/etc/fstab 文件，您可以继续使用该文件管理挂载点。

有时候用户还需要动态挂载点，比如打算访问 DVD 内容时，才临时执行挂载以便访问其中的内容，而不访问光盘时该挂载点被取消（umount），以便节约资源。传统地，人们依赖 autofs 服务来实现这种功能。

systemd 内建了自动挂载服务，无需另外安装 autofs 服务，可以直接使用 systemd 提供的自动挂载管理能力来实现 autofs 的功能。

实现事务性依赖关系管理

系统启动过程是由很多的独立工作共同组成的，这些工作之间可能存在依赖关系，比如挂载一个 NFS 文件系统必须依赖网络能够正常工作。systemd 虽然能够最大限度地并发执行很多有依赖关系的工作，但是类似“挂载 NFS”和“启动网络”这样的工作还是存在天生的先后依赖关系，无法并发执行。对于这些任务，systemd 维护一个“事务一致

性”的概念，保证所有相关的服务都可以正常启动而不会出现互相依赖，以至于死锁的情况。

与 SysV 初始化脚本兼容

和 UpStart 一样，systemd 引入了新的配置方式，对应用程序的开发也有一些新的要求。如果 systemd 想替代目前正在运行的初始化系统，就必须和现有程序兼容。任何一个 Linux 发行版都很难为了采用 systemd 而在短时间内将所有的服务代码都修改一遍。

systemd 提供了和 sysvinit 以及 LSB initscripts 兼容的特性。系统中已经存在的服务和进程无需修改。这降低了系统向 systemd 迁移的成本，使得 systemd 替换现有初始化系统成为可能。

能够对系统进行快照和恢复

systemd 支持按需启动，因此系统的运行状态是动态变化的，人们无法准确地知道系统当前运行了哪些服务。systemd 快照提供了一种将当前系统运行状态保存并恢复的能力。

比如系统当前正运行服务 A 和 B，可以用 systemd 命令行对当前系统运行状况创建快照。然后将进程 A 停止，或者做其他的任意的对系统的改变，比如启动新的进程 C。在这些改变之后，运行 systemd 的快照恢复命令，就可立即将系统恢复到快照时刻的状态，即只有服务 A，B 在运行。一个可能的应用场景是调试：比如服务器出现一些异常，为了调试用户将当前状态保存为快照，然后可以进行任意的操作，比如停止服务等等。等调试结束，恢复快照即可。

管理系统服务

systemd 提供 systemctl 命令来运行、关闭、重启、显示、启用/禁用系统服务。

sysvinit 命令和 systemd 命令

systemd 提供 systemctl 命令与 sysvinit 命令的功能类似。当前版本中依然兼容 service 和 chkconfig 命令，相关说明如表 3，但建议用 systemctl 进行系统服务管理。

表 3 sysvinit 命令和 systemd 命令的对照表

sysvinit 命令

systemd 命令

备注

service network start

```
systemctl start network.service
```

用来启动一个服务 (并不会重启现有的)。

```
service network stop
```

```
systemctl stop network.service
```

用来停止一个服务 (并不会重启现有的)。

```
service network restart
```

```
systemctl restart network.service
```

用来停止并启动一个服务。

```
service network reload
```

```
systemctl reload network.service
```

当支持时, 重新装载配置文件而不中断等待操作。

```
service network condrestart
```

```
systemctl condrestart network.service
```

如果服务正在运行那么重启它。

```
service network status
```

```
systemctl status network.service
```

检查服务的运行状态。

```
chkconfig network on
```

```
systemctl enable network.service
```

在下次启动时或满足其他触发条件时设置服务为启用。

```
chkconfig network off
```

```
systemctl disable network.service
```

在下次启动时或满足其他触发条件时设置服务为禁用。

```
chkconfig network
```

```
systemctl is-enabled network.service
```

用来检查一个服务在当前环境下被配置为启用还是禁用。

```
chkconfig --list
```

```
systemctl list-unit-files --type=service
```

输出在各个运行级别下服务的启用和禁用情况。

```
chkconfig network --list
```

```
ls /etc/systemd/system/*.wants/network.service
```

用来列出该服务在哪些运行级别下启用和禁用。

```
chkconfig network --add
```

```
systemctl daemon-reload
```

当您创建新服务文件或者变更设置时使用。

显示所有当前服务

如果您需要显示当前正在运行的服务，使用命令如下：

```
systemctl list-units --type service
```

如果您需要显示所有的服务（包括未运行的服务），需要添加-all 参数，使用命令如下：

```
systemctl list-units --type service --all
```

例如显示当前正在运行的服务，命令如下：

```
$ systemctl list-units --type service
```

UNIT	LOAD	ACTIVE	SUB	JOB	DESCRIPTION
atd.service	loaded	active	running		Deferred execution scheduler
auditd.service	loaded	active	running		Security Auditing Service
avahi-daemon.service	loaded	active	running		Avahi mDNS/DNS-SD Stack
chronyd.service	loaded	active	running		NTP client/server
crond.service	loaded	active	running		Command Scheduler
dbus.service	loaded	active	running		D-Bus System Message Bus
dracut-shutdown.service	loaded	active	exited		Restore /run/initramfs on shutdown
firewalld.service	loaded	active	running		firewalld - dynamic firewall daemon
getty@tty1.service	loaded	active	running		Getty on tty1
gssproxy.service	loaded	active	running		GSSAPI Proxy Daemon

irqbalance.service loaded active running irqbalance daemon
iscsid.service loaded activating start start Open-iSCSI

显示服务状态

如果您需要显示某个服务的状态，可执行如下命令：

systemctl status name.service

相关状态显示参数说明如表 4 所示。

表 4 状态参数说明

参数

描述

Loaded

说明服务是否被加载，并显示服务对应的绝对路径以及是否启用。

Active

说明服务是否正在运行，并显示时间节点。

Main PID

相应的系统服务的 PID 值。

CGroup

相关控制组（CGroup）的其他信息。

如果您需要鉴别某个服务是否运行，可执行如下命令：

systemctl is-active name.service

is-active 命令的返回结果如下：

表 5 is-active 命令的返回结果

状态

含义

active(running)

有一只或多只程序正在系统中执行

active(exited)

仅执行一次就正常结束的服务，目前并没有任何程序在系统中执行。举例来说，开机或者是挂载时才会进行一次的 quotaon 功能

active(waiting)

正在执行当中，不过要等待其他的事件才能继续处理。例如：打印的队列相关服务就是这种状态，虽然正在启动中，不过也需要真的有队列进来 (打印作业) 这样他才会继续唤醒打印机 服务来进行下一步打印的功能

inactive

这个服务没有运行

同样，如果您需要判断某个服务是否被启用，可执行如下命令：

```
systemctl is-enabled name.service
```

is-enabled 命令的返回结果如下：

表 6 is-enabled 命令的返回结果

状态

含义

"enabled"

已经通过 /etc/systemd/system/ 目录下的 Alias= 别名、.wants/ 或 .requires/ 软连接被永久启用。

"enabled-runtime"

已经通过 /run/systemd/system/ 目录下的 Alias= 别名、.wants/ 或 .requires/ 软连接被临时启用。

"linked"

虽然单元文件本身不在标准单元目录中，但是指向此单元文件的一个或多个软连接已经存在于 /etc/systemd/system/ 永久目录中。

"linked-runtime"

虽然单元文件本身不在标准单元目录中，但是指向此单元文件的一个或多个软连接已经存在于 /run/systemd/system/ 临时目录中。

"masked"

已经被 /etc/systemd/system/ 目录永久屏蔽(软连接指向 /dev/null 文件), 因此 start 操作会失败。

"masked-runtime"

已经被 /run/systemd/systemd/ 目录临时屏蔽(软连接指向 /dev/null 文件), 因此 start 操作会失败。

"static"

尚未被启用, 并且单元文件的 "[Install]" 小节中没有可用于 enable 命令的选项。

"indirect"

尚未被启用, 但是单元文件的 "[Install]" 小节中 Also= 选项的值列表非空(也就是列表中的某些单元可能已被启用)、或者它拥有一个不在 Also= 列表中的其他名称的别名软连接。对于模版单元来说, 表示已经启用了不同于 DefaultInstance= 的实例。

"disabled"

尚未被启用, 但是单元文件的 "[Install]" 小节中存在可用于 enable 命令的选项

"generated"

单元文件是被单元生成器动态生成的。被生成的单元文件可能并未被直接启用, 而是被单元生成器隐含的启用了。

"transient"

单元文件是被运行时 API 动态临时生成的。该临时单元可能并未被启用。

"bad"

单元文件不正确或者出现其他错误。is-enabled 不会返回此状态, 而是会显示一条出错信息。list-unit-files 命令有可能会显示此单元。

例如查看 gdm.service 服务状态, 命令如下:

```
# systemctl status gdm.service
gdm.service - GNOME Display Manager  Loaded: loaded
(/usr/lib/systemd/system/gdm.service; enabled)  Active: active (running) since Thu 2013-
10-17 17:31:23 CEST; 5min ago
Main PID: 1029 (gdm)
```

```
CGroup: /system.slice/gdm.service
├─1029 /usr/sbin/gdm
├─1037 /usr/libexec/gdm-simple-slave --display-id /org/gno...
└─1047 /usr/bin/Xorg :0 -background none -verbose -auth /r...Oct 17 17:31:23
localhost systemd[1]: Started GNOME Display Manager.
```

运行服务

如果您需要运行某个服务，请在 root 权限下执行如下命令：

```
systemctl start name.service
```

例如运行 httpd 服务，命令如下：

```
# systemctl start httpd.service
```

关闭服务

如果您需要关闭某个服务，请在 root 权限下执行如下命令：

```
systemctl stop name.service
```

例如关闭蓝牙服务，命令如下：

```
# systemctl stop bluetooth.service
```

重启服务

如果您需要重启某个服务，请在 root 权限下执行如下命令：

```
systemctl restart name.service
```

执行命令后，当前服务会被关闭，但马上重新启动。如果您指定的服务，当前处于关闭状态，执行命令后，服务也会被启动。

例如重启蓝牙服务，命令如下：

```
# systemctl restart bluetooth.service
```

启用服务

如果您需要在开机时启用某个服务，请在 root 权限下执行如下命令：

```
systemctl enable name.service
```

例如设置 httpd 服务开机时启动，命令如下：

```
# systemctl enable httpd.service
ln -s '/usr/lib/systemd/system/httpd.service' '/etc/systemd/system/multi-
user.target.wants/httpd.service'
```

禁用服务

如果您需要在开机时禁用某个服务，请在 root 权限下执行如下命令：

```
systemctl disable name.service
```

例如在开机时禁用蓝牙服务启动，命令如下：

```
# systemctl disable bluetooth.service
Removed /etc/systemd/system/bluetooth.target.wants/bluetooth.service.
Removed /etc/systemd/system/dbus-org.bluez.service.
```

改变运行级别

Target 和运行级别

systemd 用目标（target）替代了运行级别的概念，提供了更大的灵活性，如您可以继承一个已有的目标，并添加其他服务，来创建自己的目标。表 7 列举了 systemd 下的目标和常见 runlevel 的对应关系。

表 7 运行级别和 systemd 目标

运行级别

systemd 目标（target）

描述

0

runlevel0.target, poweroff.target

关闭系统。

1, s, single

runlevel1.target, rescue.target

单用户模式。

2, 4

runlevel2.target, runlevel4.target, multi-user.target

用户定义/域特定运行级别。默认等同于 3。

3

runlevel3.target, multi-user.target

多用户，非图形化。用户可以通过多个控制台或网络登录。

5

runlevel5.target, graphical.target

多用户，图形化。通常为所有运行级别 3 的服务外加图形化登录。

6

runlevel6.target, reboot.target

重启系统。

emergency

emergency.target

紧急 Shell

查看系统默认启动目标

查看当前系统默认的启动目标，命令如下：

```
systemctl get-default
```

查看当前系统所有的启动目标

查看当前系统所有的启动目标，命令如下：

```
systemctl list-units --type=target
```

改变默认目标

改变系统默认的目标，在 root 权限下执行如下命令：

```
systemctl set-default name.target
```

改变当前目标

改变当前系统的目标，在 root 权限下执行如下命令：

```
systemctl isolate name.target
```

切换到救援模式

改变当前系统为救援模式，在 root 权限下执行如下命令：

```
systemctl rescue
```

这条命令和 “systemctl isolate rescue.target” 类似。命令执行后会在串口有如下打印信息：

```
You are in rescue mode. After logging in, type "journalctl -xb" to view system logs,
"systemctl reboot" to reboot, "systemctl default" or "exit" to boot into default mode.
Give root password for maintenance
(or press Control-D to continue):
```

说明： 用户需要重启系统，从救援模式进入正常模式。

切换到紧急模式

改变当前系统为紧急模式，在 root 权限下执行如下命令：

```
systemctl emergency
```

这条命令和 “systemctl isolate emergency.target” 类似。命令执行后会在串口有如下打印信息：

```
You are in emergency mode. After logging in, type "journalctl -xb" to view system logs,
"systemctl reboot" to reboot, "systemctl default" or "exit" to boot into default mode.
Give root password for maintenance
(or press Control-D to continue):
```

说明： 用户需要重启系统，从紧急模式进入正常模式。

关闭、暂停和休眠系统

systemctl 命令

systemd 通过 systemctl 命令可以对系统进行关机、重启、休眠等一系列操作。当前仍兼容部分 Linux 常用管理命令，对应关系如表 8。建议用户使用 systemctl 命令进行操作。

表 8 命令对应关系

Linux 常用管理命令

systemctl 命令

描述

halt

systemctl halt

关闭系统

poweroff

systemctl poweroff

关闭电源

reboot

systemctl reboot

重启

关闭系统

关闭系统并下电，在 root 权限下执行如下命令：

systemctl poweroff

关闭系统但不下电机器，在 root 权限下执行如下命令：

systemctl halt

执行上述命令会给当前所有的登录用户发送一条提示消息。如果不想让 systemd 发送该消息，您可以添加 “--no-wall” 参数。具体命令如下：

systemctl --no-wall poweroff

重启系统

重启系统，在 root 权限下执行如下命令：

systemctl reboot

执行上述命令会给当前所有的登录用户发送一条提示消息。如果不想让 systemd 发送该消息，您可以添加 “--no-wall” 参数。具体命令如下：

systemctl --no-wall reboot

使系统待机

使系统待机，在 root 权限下执行如下命令：

```
systemctl suspend
```

使系统休眠

使系统休眠，在 root 权限下执行如下命令：

```
systemctl hibernate
```

使系统待机且处于休眠状态，在 root 权限下执行如下命令：

```
systemctl hybrid-sleep
```